

A machine-checked proof of the Dong–Yang classification of optimal $(n, 4)$ binary codes for BSCs

Shenghao Yang^{*,†}

Yanyan Dong^{*,†}

Abstract

We present a machine-checked Lean 4 formalization of Dong and Yang’s classification of optimal finite-length $(n, 4)$ binary block codes for binary symmetric channels. The formalization was developed mainly by feeding the paper’s proofs to an AI tool. To establish correctness, the authors verified the main theorem statements in Lean and the accepted axioms. This note discusses the corrections and simplifications made to the AI-generated formalization, and records discrepancies found in the paper during the formalization. The Lean code is available at https://github.com/shhyang/n4code_lean.

Contents

1	Introduction	2
1.1	Formalization of the Dong–Yang Classification	2
1.2	Conventions	4
2	Formalization Design of Binary Codes	5
2.1	Binary Codes in Lean	5
2.1.1	Basic Definitions of Codes	5
2.1.2	Performance of Codes	6
2.1.3	Equivalence and Code Classes	7
2.2	Statements of Main Theorems	8
2.2.1	Theorem 1 (<code>optimal_in_linear_or_class1</code>)	8
2.2.2	Theorem 2 (<code>linear_opt_*</code>)	8
2.2.3	Theorem 3 (<code>universal_strict_better_of_not_class</code>)	9
2.2.4	Theorem 4 (<code>universal_strict_better_of_not_linear</code>)	10
2.2.5	Theorem 5 (<code>optimal_codes_small_n</code>)	10

^{*}School of Science and Engineering, The Chinese University of Hong Kong, Shenzhen, Guangdong 518172, P. R. China.

[†]This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants 62501515, 62171399 and 12141108, and by the Shenzhen Fundamental Research Program under Grant JCYJ20241202124023031.

3	Proof Formalization Development	11
3.1	Axiom Audit	11
3.1.1	Trusted Axioms	11
3.1.2	decide vs. native_decide	12
3.2	Simplifications	13
3.3	Reusable Generic Core	14
4	Notable Discrepancies	15
4.1	Code Type Invariance	15
4.2	Gaps in the Column-Change Arguments	16
4.3	Proofs of Theorems 3 and 4	16
4.4	Proof of Theorem 16	17
5	Final Remarks	18

1 Introduction

Many mathematical proofs are long and complex, and it is difficult to verify their correctness by human. Machine-checked formalization promises correctness and reproducibility, but until a few years ago even medium-scale developments typically required years of effort by a dedicated team: proof size, specialized expertise, and limited tooling all acted as bottlenecks. As examples, Flyspeck [1] for the Kepler conjecture and the formal proof of the Feit–Thompson odd-order theorem [2] were each a multi-year, multi-person effort combining hand-written proof structure with extensive machine-generated case analysis. This landscape has shifted further with large language models (LLMs) that autoformalize human-written mathematics [3, 4], a task now within reach of coding agents such as OpenAI Codex.

Information and coding theory already has a substantial formalization literature in Coq, the `infotheo` library [5], and Lean 4, the `InformationTheory` namespace in Mathlib [6]. There are also standalone Lean 4 projects formalizing q -ary covering codes [7], self-dual constructions [8], and Reed–Solomon and distance machinery for succinct proofs and agreement tests [9, 10, 11, 12]. Lean-QIT [13] provides reusable infrastructure for quantum Shannon theory, including source and channel coding theorems. These efforts focus on foundational definitions and classical or structural results rather than end-to-end formalizations of contemporary research papers.

1.1 Formalization of the Dong–Yang Classification

We report on a machine-checked formalization in Lean 4 [14] of Dong and Yang’s classification of optimal finite-length $(n, 4)$ binary block codes for binary symmetric channels [15]. Its main results are classification theorems (Theorems 1–5). Theorem 1 states that for every n and ε there is an optimal $(n, 4)$ code that is linear or Class-I; Theorem 3 states that for

Table 1: Hierarchical dependence among the paper’s main results (Theorems 1–5).

Results	Technique	Ingredients
Theorem 1	inclusive covering	Theorems 6, 8, 11; Corollaries 7, 9, 10
Theorem 2	linear-code classification	Theorem 12; Corollary 13
Theorem 3	all optimal codes ($n > 3$)	Theorems 1, 2, 6, 8, 11; Lemma 14
Theorem 4	linear under Class-I hypothesis	Theorems 3, 16; Lemma 14
Theorem 5	finite classification ($2 \leq n \leq 8$)	Theorems 2, 4, 6, 8, 11, 16, 17; Lemma 15; $n = 2, 3$ base cases
Theorems 6, 8	one-column analysis	Lemmas 18, 19; Theorem 20; Corollary 21
Theorem 11	two-column analysis	Lemmas 22–27
Theorem 12	linear-code analysis	—
Theorems 16, 17	Class-I analysis	—

$n > 3$ every optimal code is equivalent to a linear, Class-I, or Class-II code; and Theorem 5 characterizes all optimal codes for $2 \leq n \leq 8$. The proofs rely on comparison techniques built around “column changes”—one- and two-column flips and the machinery that certifies improvements—and span about 27 pages in the *IEEE Trans. Inf. Theory* double-column format (Table 1).

We fed the paper’s L^AT_EX source to OpenAI Codex (models **deepseek-v4-flash**, later **deepseek-v4-pro**), which first produced definitions and a catalogue of statements to prove, each initially carrying a **sorry**, Lean’s placeholder for a missing proof. AI agents then generated proofs to discharge the **sorry**s, and the project compiled without **sorry** in about two days. The initial AI-assisted formalization required only light human intervention. We mainly reviewed the development plan and changed the LLM model to **deepseek-v4-pro** when the **deepseek-v4-flash** model failed to complete the formalization.

Although the Lean kernel checks every proof script, a **sorry**-free build is not, by itself, a complete correctness argument: one must also confirm that the Lean statements match the paper’s theorems and that the proofs depend only on trusted axioms [14, 1]. This report complements the formalization with such an audit. Section 2 introduces the code model and the Lean formulations of Theorems 1–5. For Theorems 3–5, we verified the direct paper counterparts and additionally proved stronger domination statements.

Section 3 recounts the AI-assisted development and audits the trusted axioms, including **native_decide** usage. That audit also revealed redundant proofs in the generated code. We then interact with the AI tools to simplify the proofs, which is recorded in Section 3.2. The final library comprises fourteen Lean modules totaling 37,561 lines (Table 2) and is available at https://github.com/shhyang/n4code_lean.

In addition to correctness, formalization helped clarify the paper’s proofs, which are written in natural language and are not always precise, explicit, or correct as printed. During formalization, the AI assistant often exposed these gaps: it corrected errors, sharpened imprecise statements, and made implicit steps explicit. We catalogue the discrepancies in Section 4.

Table 2: Modules, sizes, and content as of the checked and improved formalization.

Module	Lines	Content
Definitions.lean	449	Codes as columns; Hamming/ λ ; classes; equivalence
Basic.lean	610	Weight/distance lemmas; count identities; α basics
Performance.lean	449	Monotonicity of λ ; invariance under equivalence
Comparison.lean	101	Generic comparison engine (Lemma 18: <code>compare_bij</code>)
Compare.lean	1479	One-column engine (Lemmas 19–20, Theorem 20, Corollary 21)
ZeroColumn.lean	2933	Replacing type-0 columns (Theorem 6, Corollary 7)
TwoColumn.lean	3361	Two-column Z -regions and Theorem 11
Reduction.lean	5696	One-bit flip and reductions (Theorems 1, 8; Corollaries 9–10; Lemmas 14–15)
Linear.lean	10775	Linear comparison and classification (Theorems 2, 12; Corollary 13; Lemma 28)
ClassI.lean	6930	Class-I analysis (Theorems 16–17)
FiniteN.lean	2352	Finite classification for $2 \leq n \leq 8$ (Theorem 5)
Nbig.lean	2246	Covering for $n > 3$ (Theorems 3–4)
Statements.lean	100	Paper-label catalogue of theorem statements
AxiomCheck.lean	80	<code>#print axioms</code> audit of headline theorems
Total	37,561	

Within the Lean 4 ecosystem, `mathlib` [6] already provides Hamming distance and linear codes over finite fields (`Mathlib/CodingTheory/`). Although we rely on `mathlib` for general infrastructure, we do not build on its coding-theory library, which targets the construction and analysis of linear codes via generator matrices, subspaces, and minimum distance. The Dong–Yang proofs instead use column types, equivalence under column changes, and nonlinear codes, so we built a complementary framework for Hamming distance, distance distributions, and maximum-likelihood decoding on $(n, 4)$ codes.

1.2 Conventions

We adopt the following conventions unless stated otherwise:

- Theorems, lemmas, corollaries, and displayed equations are cited by their numbers in [15].
- The paper indexes codewords by $c_1, \dots, c_4$; Lean uses `row0 C–row3 C` (i.e. rows 0–3). We use the two conventions interchangeably.
- Inline identifiers such as `lambda C ε` are rendered from the source; module names appear in `typewriter` (e.g. [Reduction.lean](#)); line ranges link to the corresponding source on GitHub.

2 Formalization Design of Binary Codes

This section shows how the main theorems are formulated in Lean. [Section 2.1](#) introduces the code model; [Section 2.2](#) gives the Lean statements of Theorems 1–5 for comparison with the paper.

Lean 4 [14] is a dependently typed proof assistant: mathematical objects and propositions are types, and a proof is a program whose type is the theorem statement. Lean’s trusted kernel checks these typing judgments. The development lives in the `N4Code` namespace and builds on `mathlib` [6] for finite types (`Fin n` for bounded indices, `Finset` for finite sets), big sums written as $\sum i \in s, f i$, parity (`Even/Odd`), and real arithmetic ($\mathbb{R}$). We use $\mathbb{N}$ and $\mathbb{R}$ for the natural and real numbers; `A → B` for functions; `∀` and `∃` for quantifiers; and `Prop` for propositions. An expression `t : T` means that `t` has type `T`; `def` and `abbrev` introduce definitions, while `lemma` and `theorem` mark proved statements.

2.1 Binary Codes in Lean

The paper [15] studies optimal finite-length binary block codes of size four for a binary symmetric channel (BSC) with crossover probability ε . A code $C \subseteq \{0, 1\}^n$ of blocklength n is optimal if it maximizes the correct-decoding probability $\lambda_C(\varepsilon)$ under maximum-likelihood decoding.

Mathematically, a binary $(n, 4)$ code is a set of four distinct words $c_1, \dots, c_4 \in \{0, 1\}^n$. The paper indexes rows by 1, 2, 3, 4 and Lean by 0–3; we use the two conventions interchangeably.

2.1.1 Basic Definitions of Codes

The paper’s proofs manipulate columns of the $4 \times n$ matrix whose rows are those codewords, so our formalization takes columns as the primary object:

```
abbrev Word (n : ℕ) := Fin n → Bool
abbrev Column := Fin 4 → Bool
abbrev Code (n : ℕ) := Fin n → Column
```

Here `Fin k` is `mathlib`’s type $\{0, 1, \dots, k-1\}$; a `Column` is four bits indexed by `Fin 4`; and `Code n` is an $(n, 4)$ code given by its n columns. The j -th codeword is `row C j : Word n` ($j = 0, 1, 2, 3$), also abbreviated `row0 C, ..., row3 C`.

The type `Code n` is slightly more general than the notion of an $(n, 4)$ code used in [15]. In the paper, an $(n, 4)$ code is a four-element subset of $\{0, 1\}^n$, and hence its four codewords are pairwise distinct. In contrast, `Code n` represents an arbitrary $4 \times n$ binary matrix and therefore may also have repeated rows. We deliberately work in this enlarged space because it simplifies the intermediate column transformations used throughout the formalization. Pairwise distinctness of the four rows is recorded separately in Lean by the predicate `DistinctRows`.

Weight and distance on words can be defined as follows:

```
def bitXor {n : ℕ} (x y : Word n) : Word n :=
  fun i => Bool.xor (x i) (y i)
def hammingWeight {n : ℕ} (x : Word n) : ℕ :=
  ∑ i ∈ Finset.univ, if x i = true then 1 else 0
def hammingDist {n : ℕ} (x y : Word n) : ℕ :=
  hammingWeight (bitXor x y)
```

Distance from a received word y to codeword c_j , and to the code, are `dRow` and `dCode` respectively.

```
def dRow {n : ℕ} (C : Code n) (j : Fin 4) (y : Word n) : ℕ :=
  hammingDist (row C j) y
def dCode {n : ℕ} (C : Code n) (y : Word n) : ℕ :=
  min (hammingDist (row0 C) y)
    (min (hammingDist (row1 C) y)
      (min (hammingDist (row2 C) y) (hammingDist (row3 C) y)))
```

2.1.2 Performance of Codes

On a BSC with crossover $\varepsilon \in (0, 1/2)$, the average correct-decoding probability under ML decoding is

$$\lambda_C(\varepsilon) := \frac{1}{4} \sum_{y \in \{0,1\}^n} (1 - \varepsilon)^{n-d_C(y)} \varepsilon^{d_C(y)},$$

formalized as `lambda C ε` ([Definitions.lean](#), ll. 110–111). The per-word contribution

$$w_{n,\varepsilon}(d) := (1 - \varepsilon)^{n-d} \varepsilon^d,$$

is formalized as `weight n ε d` ([Performance.lean](#), l. 23). With the counts $\alpha_C(d)$ of the number of y with $d_C(y) = d$, we have

$$\lambda_C(\varepsilon) = \frac{1}{4} \sum_d \alpha_C(d) w_{n,\varepsilon}(d).$$

Optimality and universal comparison are then

```
def OptimalAt {n : ℕ} (C : Code n) (ε : ℝ) : Prop :=
  ∀ D : Code n, lambda C ε ≥ lambda D ε
def UniversalBetter {n : ℕ} (C1 C2 : Code n) : Prop :=
  ∀ ε : ℝ, 0 < ε → ε < 1 / 2 → lambda C1 ε ≥ lambda C2 ε
def UniversalStrictBetter {n : ℕ} (C1 C2 : Code n) : Prop :=
  ∀ ε : ℝ, 0 < ε → ε < 1 / 2 → lambda C1 ε > lambda C2 ε
def UniversalEqual {n : ℕ} (C1 C2 : Code n) : Prop :=
  ∀ ε : ℝ, 0 < ε → ε < 1 / 2 → lambda C1 ε = lambda C2 ε
```

so `OptimalAt C ε` is optimality at a fixed ε , while `UniversalBetter C' C` means $\lambda_{C'}(\varepsilon) \geq \lambda_C(\varepsilon)$ for every $\varepsilon \in (0, 1/2)$ (with a strict variant `UniversalStrictBetter` and an equality variant `UniversalEqual`).

Although `OptimalAt` quantifies over the enlarged type `Code n`, this does not change the optimal-code problem for $n \geq 2$. We prove that every code matrix with repeated rows is universally strictly dominated by one with pairwise distinct rows. Consequently, every `OptimalAt` code has pairwise distinct rows. Thus the additional objects admitted by `Code n` can occur only as intermediate nonoptimal matrices, and the optimal elements of the Lean model coincide with genuine $(n, 4)$ codes as defined in [15].

2.1.3 Equivalence and Code Classes

Two codes are equivalent if one is obtained from the other by row permutation, column permutation, and flipping all bits in some columns:

```
def Equivalent {n : ℕ} (C C' : Code n) : Prop :=
  ∃ p : Equiv (Fin 4) (Fin 4), ∃ p : Equiv (Fin n) (Fin n),
  ∃ f : Fin n → Bool,
  ∀ t : Fin n,
    C' (p t) = rowPermute p
      (if f t then flipCol (C t) else C t)
```

Equivalence preserves λ_C , so classification statements are naturally up to `Equivalent`.

Each column has a type given by reading its four bits as a binary integer in $\{0, \dots, 15\}$. Note that the paper's column-type bit order is the reverse of this natural row indexing: Bit j of a column is row $(4 - j)$'s entry, with row 0 most significant. Named columns `col0`, `col1`, `col3`, `col5`, `col6`, `col7` are the standard representatives of those types. The paper defines $|i|_C$ (or $|i|$) as the number of columns of type i in a code C :

```
def colVal (c : Column) : ℕ :=
  ∑ j : Fin 4, if c j then 2 ^ (3 - j.val) else 0
def count {n : ℕ} (C : Code n) (i : ℕ) : ℕ :=
  ∑ t : Fin n, if colVal (C t) = i then 1 else 0
```

The sum of counts over a set s of types is `totalCounts C s`. Linear codes and the three nonlinear classes (Class-I, Class-II, Class-III) used in the reductions are predicates on these counts:

```
def IsLinear {n : ℕ} (C : Code n) : Prop :=
  (∀ t : Fin n, colVal (C t) = 0 ∨ colVal (C t) = 3 ∨
    colVal (C t) = 5 ∨ colVal (C t) = 6) ∧
  ((count C 3 > 0 ∧ count C 5 > 0) ∨
   (count C 3 > 0 ∧ count C 6 > 0) ∨
   (count C 5 > 0 ∧ count C 6 > 0))
```

```

def ClassI {n : ℕ} (C : Code n) : Prop :=
  Odd (count C 1) ∧
  ((Even (count C 3) ∧ Even (count C 5) ∧ Even (count C 6)) ∨
   (Odd (count C 3) ∧ Odd (count C 5) ∧ Odd (count C 6))) ∧
  totalCounts C {1, 3, 5, 6} = n
def ClassII {n : ℕ} (C : Code n) : Prop :=
  count C 1 > 0 ∧ totalCounts C {1, 3, 5, 6} = n ∧
  ((Even (count C 1) ∧ Even (count C 3) ∧
   Odd (count C 5) ∧ Odd (count C 6)) ∨
   (Even (count C 1) ∧ Odd (count C 3) ∧
   Even (count C 5) ∧ Even (count C 6)))
def ClassIII {n : ℕ} (C : Code n) : Prop :=
  totalCounts C {1, 3, 5, 6, 7} = n ∧
  ((count C 1 = 1 ∧ count C 7 = 1 ∧ count C 6 = 0 ∧
   Even (count C 3) ∧ Odd (count C 5)) ∨
   (count C 1 = 1 ∧ count C 5 = 0 ∧ count C 7 = 0 ∧
   Odd (count C 3) ∧ Odd (count C 6)))

```

In short: `IsLinear` allows only types 0, 3, 5, 6 with at least two of $|3|$, $|5|$, $|6|$ positive; `ClassI`/`ClassII` use only $\{1, 3, 5, 6\}$ under the stated parity patterns; and `ClassIII` additionally allows type 7 under one of two count patterns. The canonical linear representative with $|3| = n_3$, $|5| = n_5$, $|6| = n_6$ is `linearCode n3 n5 n6`.

2.2 Statements of Main Theorems

Using the vocabulary of [Section 2.1](#), the five headline classification theorems of [\[15, §2\]](#) are stated as follows.

2.2.1 Theorem 1 (`optimal_in_linear_or_class1`)

For every $n \geq 2$ and every $\varepsilon \in (0, 1/2)$ there exists a code that is optimal at ε and is linear or Class-I. The theorem is stated in [Reduction.lean](#), ll. 5686–5688:

```

theorem optimal_in_linear_or_class1 (n : ℕ) (hn : 2 ≤ n) :
  ∀ ε : ℝ, 0 < ε → ε < 1 / 2 →
    ∃ C : Code n, (IsLinear C ∨ ClassI C) ∧ OptimalAt C ε

```

The hypothesis $n \geq 2$ makes the paper’s standing blocklength assumption explicit; the conclusion is per- ε .

2.2.2 Theorem 2 (`linear_opt_*`)

Theorem 2 of the paper classifies optimal linear $(n, 4)$ codes by $n \bmod 3$ and states that each listed code is universally strictly better than every other non-equivalent linear code. The formalization gives four declarations in [Linear.lean](#), one per residue class:

- [linear_opt_residue2](#) (ll. 9835–9838): $n = 3k - 1$, $C(k, k, k - 1)$ is optimal;
- [linear_opt_n3](#) (ll. 10331–10336): $n = 3$, C_A , $C(1, 1, 1)$, $C(1, 2, 0)$ are optimal;
- [linear_opt_residue0](#) (ll. 10625–10633): $n = 3k$ ($k \geq 2$), $C(k + 1, k + 1, k - 2)$ and $C(k + 1, k, k - 1)$ are optimal;
- [linear_opt_residue1](#) (ll. 10715–10723): $n = 3k + 1$, $C(k + 1, k, k)$ and $C(k + 2, k, k - 1)$ are optimal.

Each concludes **UniversalStrictBetter** (strict universal optimality).

The paper’s closing domination sentence is imprecise when several codes are listed for the same n ($n = 3$; $n = 3k$, $k \geq 2$; $n = 3k + 1$): they are pairwise non-equivalent yet can have equal λ (e.g. at $n = 3$, $\lambda(C(1, 1, 1)) = \lambda(C(1, 2, 0)) = \lambda(C_A)$), so no listed code strictly dominates all the others. The claim is correct only if “not equivalent to them” is read collectively: a competitor D must be non-equivalent to *every* listed optimum before *each* listed code strictly dominates D . The $n = 3k - 1$ case lists a single optimum $C(k, k, k - 1)$ and needs no such collective reading.

The $n = 3$ declaration is representative; it requires a separate non-equivalence hypothesis for each listed code:

```

theorem linear_opt_n3 :
  ∀ D : Code 3, IsLinear D →
    ¬ Equivalent CA D → ¬ Equivalent (linearCode 1 1 1) D →
    ¬ Equivalent (linearCode 1 2 0) D →
    UniversalStrictBetter CA D ∧
    UniversalStrictBetter (linearCode 1 1 1) D ∧
    UniversalStrictBetter (linearCode 1 2 0) D

```

Here [CA](#) is the code with columns $(3, 5, 0)$.

The same pattern appears in [linear_opt_residue0](#) and [linear_opt_residue1](#) (two listed optima, two non-equivalence hypotheses); [linear_opt_residue2](#) states the single-optimum case directly. All four are proved in [Linear.lean](#).

2.2.3 Theorem 3 (universal_strict_better_of_not_class)

In the paper, Theorem 3 is stated as follows:

For $(n, 4)$ codes with $n > 3$, the following property universally holds: the set comprising the optimal linear codes, Class-I codes, Class-II codes, and their equivalent codes contains all the optimal codes.

The initial Lean formalization used a per- ε classification instead: [optimal_equivalent_linear_class1_class2](#) ([Nbig.lean](#), ll. 2206–2210):

```

theorem optimal_equivalent_linear_class1_class2 (n : ℕ) (hn : n > 3) :
  ∀ ε : ℝ, 0 < ε → ε < 1 / 2 → ∀ C : Code n,
    OptimalAt C ε →

```

```

    ∃ C' : Code n, Equivalent C C' ∧
      (IsLinear C' ∨ ClassI C' ∨ ClassII C')

```

However, this formalization does not accurately state the meaning of “universally” in the theorem in the paper.

Comparing the paper and Lean proofs led us to a sharper universal statement: any code outside the three classes has another code that is strictly better at *every* crossover probability. We formalize this as `universal_strict_better_of_not_class` ([Nbig.lean](#), ll. 2193–2199):

```

theorem universal_strict_better_of_not_class (n : ℕ) (hn : n > 3) (C : Code n)
  (hnot : ∀ C' : Code n, Equivalent C C' →
    ¬ (IsLinear C' ∨ ClassI C' ∨ ClassII C')) :
  ∃ D : Code n, UniversalStrictBetter D C

```

The theorem `optimal_equivalent_linear_class1_class2` is a corollary of this one.

2.2.4 Theorem 4 (`universal_strict_better_of_not_linear`)

Theorem 4 gives a sufficient condition under which all optimal $(n, 4)$ codes are equivalent to linear codes. As with Theorem 3, we use a strengthened universal form: under the Class-I replacement condition, any code not equivalent to a linear code admits another that is universally strictly better. The formal statement is `universal_strict_better_of_not_linear` ([Nbig.lean](#), ll. 2218–2224):

```

theorem universal_strict_better_of_not_linear (n : ℕ) (hn : n > 3)
  (hcond : ∀ C : Code n, ClassI C → count C 1 ≥ 3 →
    ∀ t : Fin n, C t = col1 →
      UniversalBetter (replaceColumn C t (argminType C)) C) :
  ∀ C : Code n,
    (∀ C' : Code n, Equivalent C C' → ¬ IsLinear C') →
      ∃ D : Code n, UniversalStrictBetter D C

```

Here, a one-column change is `replaceColumn C t s'`; `argminType C` is the type among $\{3, 5, 6\}$ of minimal count (ties broken in that order).

When Theorem 4’s hypothesis holds, Theorem 2 implies that optimal linear codes are universally strictly better than every non-equivalent competitor.

2.2.5 Theorem 5 (`optimal_codes_small_n`)

Theorem 5 lists all optimal $(n, 4)$ codes for $2 \leq n \leq 8$. The strengthened Lean statement is `optimal_codes_small_n` ([FiniteN.lean](#), ll. 2336–2342): for $n \neq 3$, a code not equivalent to a linear code has a universally strictly better code; for $n = 3$, a code not equivalent to one of the five codes has a universally strictly better code.

For $4 \leq n \leq 8$, Theorem 5 follows from verifying Theorem 4’s hypothesis. The Lean statement is `n4to8_strict_better_of_not_linear` (`FiniteN.lean`, ll. 2220–2225): if C is not equivalent to a linear code, then some code is universally strictly better than C .

```
theorem n4to8_strict_better_of_not_linear (n : ℕ) (hn4 : n > 3) (hn8 : n ≤ 8) :
  ∀ C : Code n,
    (∀ C' : Code n, Equivalent C C' → ¬ IsLinear C') →
      ∃ D : Code n, UniversalStrictBetter D C
```

For $n = 2$, Theorem 4 does not apply because it requires $n > 3$. The Lean statement is `n2_strict_better_of_not_linear` (`FiniteN.lean`, l. 260).

For $n = 3$, the Lean statement is `n3_strict_better_of_not_InOptimal3` (`FiniteN.lean`, ll. 2253–2255): if a code is not equivalent to one of the five codes in `InOptimal3` (`code135`, `code136`, `CA`, `linearCode 1 1 1`, `linearCode 1 2 0`), then some code is universally strictly better than it. The converse is also formalized by `n3_representatives_optimal`: each of the five representatives is universally optimal among all $(3, 4)$ codes.

3 Proof Formalization Development

The initial code produced by the assistant consisted of definitions together with a catalogue of statements to prove (each carrying a `sorry`). AI agents then worked in parallel to discharge those `sorry`s. Development was tracked in a git repository, with each agent operating on its own clone (worktree) of the codebase. All `sorry`s were removed within approximately 48 hours, with only light human intervention. The resulting library contains no `sorry`, `admit`, or `axiom` declarations; every headline theorem is proved, and `lake build` succeeds.

A `sorry`-free build is not enough by itself: the Lean kernel checks proofs relative to the axioms they depend on, so those axioms must be audited as well. [Section 3.1](#) records how the development was made to depend only on Lean’s standard base axioms: the `native_decide` trust axioms that appeared in an earlier draft were eliminated by replacing them with `decide`.

During the audit, we also found redundant proofs—for example, distances between code-words were computed differently in different modules. We guided the AI to rewrite them for consistency and brevity; the results are summarized in [Section 3.2](#) and do not change any theorem statement. The simplified formalization totals 37,561 lines across fourteen modules; [Table 2](#) gives the size and main content of each.

Finally, we extract the generic core of the formalization that is independent of this specific $(n, 4)$ classification problem. We summarize the core in [Section 3.3](#).

3.1 Axiom Audit

3.1.1 Trusted Axioms

A successful build with no `sorry` is not enough: the proofs must depend only on an explicit allowlist of axioms. The module `N4Code/AxiomCheck.lean` runs `#print axioms` on every head-

line theorem, and `scripts/axioms_check.sh` checks that each axiom list is contained in the allowlist (any other axiom—in particular `sorryAx`—fails the check).

The first class of permitted axioms is Lean’s standard base axioms `propext`, `Quot.sound`, and `Classical.choice`:

- `propext` (propositional extensionality): equivalent propositions are equal, i.e. $(P \leftrightarrow Q) \rightarrow P = Q$;
- `Quot.sound`: if $x \sim y$ under a relation used to form a quotient type, then the images of x and y in the quotient are equal;
- `Classical.choice`: every nonempty type has an element (Hilbert’s ε -operator), which yields classical logic (including the law of excluded middle).

These are the default axioms of Lean’s mathematics library; they are not specific to this formalization.

3.1.2 From `native_decide` to `decide`

A proposition P is *decidable* in Lean when an instance of `Decidable P` supplies either a proof of P or a proof of $\neg P$. The function `decide` maps such a proposition to a boolean, and the tactic `decide` closes a goal P by reducing `decide P` to `true` inside the Lean kernel. Since the reduction is pure kernel computation, this path adds no axiom beyond the standard base axioms.

The tactic `native_decide` takes a different path: it compiles the same decision procedure to native code, executes it in the Lean runtime, and—if the result is `true`—records a *trust axiom* asserting the theorem. In `#print axioms` these trust facts appear as per-lemma names of the form `<lemma>._native.native_decide.ax_*`, for example `two_zero_columns._native.native_decide.ax_1_10`. Accepting such axioms makes the Lean compiler and runtime part of the trusted computing base.

In the released version we eliminate this extra trust assumption entirely. Every `native_decide` in `N4Code` is replaced by `decide`; the finite checks—concrete column-type arithmetic (`colVal`, `testBit`), exhaustive searches over $\{0, 1\}^n$ for $n \leq 3$, and equivalence witnesses on fixed matrices—are small enough to be closed by kernel reduction. Where the default kernel recursion limit was too small, the relevant module sets `set_option maxRecDepth 1000000`. After the replacement, `N4Code/AxiomCheck.lean` reports only `propext`, `Classical.choice`, and `Quot.sound` for every headline theorem, and `scripts/axioms_check.sh` no longer permits `Lean.ofReduceBool` or `*._native.native_decide.ax_*`.

The former distribution of `native_decide` declarations, retained only as development history, is shown in Table 3.

In the earlier draft, every one of the 215 `native_decide`-bearing declarations (lemmas, theorems, and the relevant `example` blocks) carried a marker line immediately above its docstring:

```
-- native_decide: Contentful · n=3 · checked 2026-08-24
```

Table 3: Former declarations carrying `native_decide`, by module (all replaced by `decide` in the released version).

Module	Declarations	Module	Declarations
Reduction	91	ZeroColumn	10
FiniteN	33	ClassI	17
Nbig	24	Definitions	7
TwoColumn	10	Basic	2
Linear	17	Performance	4

The marker records the audit group (`Mechanical` or `Contentful`), the block length ($n = 2$, $n = 3$, or `any` when the statement is generic), and the date of the check. After the replacement these marker lines are retained but now read `-- decide::`; the same declarations can be enumerated with

```
rg -- "-- decide:" N4Code/
```

and checked, per marker, that the surrounding declaration is of the stated group and block length.

3.2 Simplifications

These simplifications are internal proof cleanup; no theorem statement or paper correspondence changed.

- **Row-distance lemmas.** While re-examining the formalization, we replaced the repeated row-pair distance instances in `Reduction.lean` and `Nbig.lean` with one general lemma `hammingDist_rows_of_types`: for a code whose column types lie in a finite set S , the Hamming distance between rows i and j is the weighted sum over S of the type counts whose i -th and j -th bits differ. The specialized $\{1, 3, 5, 6\}$, $\{1, 3, 5, 7\}$, and $\{3, 5, 6\}$ statements are now short corollaries; the four full-type-set formulas `hammingDist_row0_row2_eq`, `hammingDist_row1_row2_eq`, `hammingDist_row2_row3_eq`, and `hammingDist_row1_row3_eq` for `Columns07` codes are one-line corollaries with $S = \text{Finset.Icc } 0 \ 7$. The lemma, together with `sum_indicator_of_types`, now lives in `Basic.lean`; the redundant `sum_colVal_indicator` helper in `TwoColumn.lean` is removed.
- **Count-replacement lemmas.** The three general column-replacement lemmas `count_replace_dec`, `count_replace_inc`, and `count_replace_eq` now live in `Compare.lean` next to `sum_split_at`. The 1-to-3 instances in `Reduction.lean` and the zero-column instances in `Linear.lean` and `ZeroColumn.lean` are short corollaries of this family, instead of re-deriving the same sum-splitting argument.
- **Counts across involutive relabellings.** The eight `count_swap12Code_*` and `count_swap36Code_*` lemmas each re-derived the same per-type counting argument. They are now one-line corollaries of `count_involution_map` in `Basic.lean` (counts transport across

an involutive type relabelling), together with the `swapVal12` / `swapVal36` involutions and the `colVal_swap12Code` / `colVal_swap36Code` type maps. The same general fact now also absorbs `count_swapRows01Code` and `count_swapRows02Code` in `Reduction.lean`.

- **Positive-count witnesses.** The three lemmas `exists_col0_of_count_pos`, `exists_col1_of_count_pos`, and `exists_col7_of_count_pos` were specialized copies of `count_pos_iff_exists`; they are now one-liners combining that general fact with the matching `colVal_eq*_iff_col*` lemma.
- **Class-I hypothesis reuse for the 3 6 swap.** The `col6` branches of `class1_one` and `class1_min` previously re-derived the Class-I hypotheses (`htypes`, the parity conditions, and `totalCounts`) on `swap36Code c` by hand. A new `ClassI_swap36Code` lemma supplies the Class-I property of the swapped code, and `classI_hyps` now unpacks it, so those branches work directly from `ClassI c`. The now-unused `types_swap36Code` was removed.
- **Linear block counting.** The three `linear_count_3/5/6` lemmas counted the blocks of a linear code, with `linear_count_5` and `linear_count_6` each re-proving the same offset bijection. We added the general `card_fin_interval` and `card_fin_suffix` helpers (next to the existing prefix helper `card_fin_lt`), so the middle and suffix counts are now one-liners.
- **Unified `colVal` iff lemmas.** We consolidated the six `colVal_eq*_iff_col*` lemmas into two general facts, `colVal_col0fNat` and `colVal_eq_iff_col0fNat`. The named-column instances for types 0, 1, 3, 5, 6, 7 are now short corollaries.

3.3 Reusable Generic Core

The simplification pass described above reworked the development around small, reusable lemmas in addition to shortening proofs. These components form two tiers.

- *Generic infrastructure.* The foundational modules `Definitions`, `Basic`, `Performance`, and `Comparison` (together with `Compare` for the one-column machinery) provide, without any Class-I/II/III dependence, the Hamming geometry and code model, the type-number bijection (`colVal_col0fNat` and `colVal_eq_iff_col0fNat`), equivalence invariance and transport (`dRow_equiv`, `dCode_equiv`, `alpha_equiv`, `lambda_equiv`, `universalEqual_of_equiv`, `universalStrictBetter_of_equiv`), the BSC weight and order machinery, and the generic comparison engine (`compare_bij`). The simplification also extracted the general counting and grouping facts `sum_split_at`, `count_replace_dec`, `count_replace_inc`, `count_replace_eq`, `sum_indicator_of_types`, `hammingDist_rows_of_types`, and `count_involution_map`, together with the `card_fin_lt`, `card_fin_interval`, and `card_fin_suffix` block-counting helpers.
- *Paper-specific but factored.* Components tied to the 4-column Class-I setting—such as `swapVal12`, `swapVal36`, `colVal_swap12Code`, `colVal_swap36Code`, `Equivalent_swap36Code`, `ClassI_swap12Code`, `ClassI_swap36Code`, and `classI_hyps`—are not generic, but they are now factored so the classification proofs reuse them instead of repeating ad-hoc case analyses.

The remaining modules build the classification on the generic tier and depend only on it.

Table 4: Summary of discrepancies between the published paper and the formalization.

Location	Type	Issue
§II-A, type invariance	gap	Multiset invariance stated, not proved
Cor. 10	proof gap	Thm. 8 only for $w(c_3 \oplus c_4)$ even
Thm. 16, $n = 3$	strength	Vacuous for genuine codes; Lean adds DistinctRows
Thm. 16, argmin	proof gap	$ 5 , 6 $ minimizer cases incomplete
Thm. 4	proof gap	“Equivalent to Class-II” step
Lem. 15 Case 1	labeling	“Class-II-b” should be Class-III-b
Thm. 3 proof	imprecision	Linear image: one of $ 5 , 6 $ zero
Thm. 16 proof, $n = 2$	infeasible-case error	The $n = 2$ case is impossible under the Class-I parity conditions
Thms. 8, 11	convention	Equality characterizations need types 0–7
Thm. 12, eq. (244)	convention	$\mathcal{Y}_3$ converse needs consistency
Thm. 6(2)	convention	0–15 column identification

We expect the generic infrastructure—the distance-distribution machinery, the cumulative-sum criterion (Corollary 21), and the equivalence-invariance machinery—to be reusable by future formalizations in information theory, complementing mathlib’s existing coding-theory library. The machine-checked paper-to-Lean dictionary and the consistency tooling keep the correspondence auditable as the generic core is put to new uses.

4 Notable Discrepancies

During verification we found that the formalization follows the paper closely overall, but Lean code is not a direct translation of the prose: formalization demands precise statements, and the paper’s natural language is not always precise, explicit, or correct. The subsections below and Table 4 catalogue the main issues by theme; each subsection gives fuller detail.

4.1 Code Types Under Equivalence

The paper states that code types are invariant under equivalence: no equivalent code should belong to two different classes among linear, Class-I, and Class-II. It offers no formal proof, but appeals to the following facts. In Section II-C, eqs. (21)–(22) claim that for equivalent codes C and C' with columns only in types 0–7,

$$\{|1|, |2|, |4|, |7|\}_C = \{|1|, |2|, |4|, |7|\}_{C'} \quad \text{and} \quad \{|3|, |5|, |6|\}_C = \{|3|, |5|, |6|\}_{C'}$$

as multisets. For example, flipping a type-1 column yields type 14, and swapping rows 1 and 4 turns that into type 7, so a code with a type-1 column can be equivalent to one with a type-7 column, while preserving the multiset $\{|3|, |5|, |6|\}$.

Lean does not formalize the paper’s multiset invariance directly; instead, equivalence preserves the parity of column Hamming weights (`allEvenWeights_equiv`). Since linear codes have only even-weight columns (`IsLinear_all_even`) while Class-I, Class-II, and Class-III codes each have a type-1 column, the lemmas `no_cross_class_linear_class1`, `no_cross_class_linear_class2`, and `no_cross_class_linear_class3` in `Performance.lean` show that no linear code is equivalent to any of these three classes. Separation among Class-I, Class-II, and Class-III is not proved this way and would need a finer invariant than column-weight parity alone.

4.2 Gaps in the Column-Change Arguments

Corollary 10 shows that a non-Class-I nonlinear code with $|1| + |3| + |5| + |6| = n$ admits a linear or Class-I code with smaller $|1|$. The proof observes that at least one of

$$w(c_1 \oplus c_4) = |1| + |3| + |5|, \quad w(c_3 \oplus c_4) = |1| + |5| + |6|, \quad w(c_2 \oplus c_4) = |1| + |3| + |6|$$

is even and invokes Theorem 8, claiming a better code with $|1|$ reduced by one while preserving $|1| + |3| + |5| + |6| = n$. Theorem 8, however, is proved only when $w(c_3 \oplus c_4)$ is even; extensions to the other row pairs appear only in the remark after Theorem 8.

Lean closes this gap formally for the $w(c_1 \oplus c_4)$ - and $w(c_2 \oplus c_4)$ -even cases that Theorem 8 leaves to its remark. The latter is handled by `one_bit_flip_1_to_5`; the former uses `swapRow_s01Code` (rows $c_1 \leftrightarrow c_2$, plus column flips keeping types 0–7) to reduce to the $w(c_2 \oplus c_4)$ -even, $1 \rightarrow 5$ flip on an equivalent code while preserving `totalCounts` over $\{1, 3, 5, 6\}$. `onepo_step` and `descent_to_linear_or_class1` in `Reduction.lean` assemble these into the full Corollary 10 formal proof.

4.3 Proofs of Theorems 3 and 4

The proofs of Theorems 3 and 4 contain gaps that Lean closes explicitly.

In the Class-III branch of Theorem 3, the paper says the linear image C' can be chosen “without type $\langle 5 \rangle$ or $\langle 6 \rangle$ columns”. The intended reading is that exactly one of $|5|, |6|$ is zero: $|6| = 0$ after the Class-III-a map $(1, 7) \rightarrow (3, 5)$ and $|5| = 0$ after the Class-III-b map $1 \rightarrow 3$, with the other two linear types positive. If both counts were zero, C' would be all type 3. Lean proves this precise form in `classIII_not_optimal`.

In Lemma 15, Case 1, the three equality conditions of Theorem 8 are cited as follows: (i) $w(c_1 \oplus c_3)$ and $w(c_2 \oplus c_3)$ both odd (then C is Class-II); (ii) $|1| = 1$, $|5| = 0$, $|3|$ and $|6|$ odd (where the paper says “Class-II-b”); (iii) $|1| = 1$, $|6| = 0$, $|3|$ and $|5|$ odd (where the paper says “equivalent to Class-II-b”). Class-II-b requires $|1|$ even, so neither (ii) nor (iii) ($|1| = 1$) is Class-II-b; under Case 1 ($|2| = |4| = |7| = 0$), (ii) is Class-III-b and (iii) becomes Class-III-b after `swapRows01Code` exchanges types $5 \leftrightarrow 6$. Lemma 15’s conclusion is unaffected; only the two labels should read “Class-III-b”.

Theorem 4 drives the $|1| \geq 3$ descent for Class-I codes: replace one type-1 column by the argmin type $s \in \{3, 5, 6\}$ to obtain C' with $|1|_{C'} = |1| - 1$, then apply Lemma 14 to reach

Class-I with $|1|$ reduced by 2. The paper asserts that C' “is equivalent to a Class-II code” before invoking Lemma 14. Lean closes this step in `classI_descend_one` (`Nbig.lean`), with one branch per argmin type. For $s = 3$, C' is Class-II directly. For $s = 5$, C' is not Class-II, but `swap12Code` ($c_2 \leftrightarrow c_3$) is a code equivalence mapping C' to Class-II. For $s = 6$, the needed $3 \leftrightarrow 6$ relabelling is likewise realizable by `Equivalent_swap36Code` (flip type-3/6 columns and swap $c_1 \leftrightarrow c_3$, as in `swapRows02Code`). In each branch the role-swapped code $\tilde{C}$ is Class-II, Lemma 14 applies, and the $|1| \rightarrow |1| - 2$ descent of Theorem 4 follows.

4.4 Proof of Theorem 16

Theorem 16 states that for a Class-I code with $|1| = 1$, replacing the type-1 column by the argmin type among $\{3, 5, 6\}$ yields $\lambda_{C'} > \lambda_C$ when $n \neq 3$ and $\lambda_{C'} = \lambda_C$ when $n = 3$.

First, for a genuine $(n, 4)$ code with pairwise distinct codewords, no Class-I code with $|1| = 1$ exists at $n = 3$, so the $n = 3$ equality clause is vacuous. Lean’s `Code` type does not enforce distinct rows; for example, a code with one type-1 column and two type-3 columns is Class-I with $c_1 = c_2$. Here the argmin type is 5, so replacing the type-1 column yields the linear code $C(2, 1, 0)$, which is strictly better. The formalization therefore adds `DistinctRows` to `class1_one` (the Lean counterpart of Theorem 16).

Second, the proof of Theorem 16 states that for $2 \leq n \leq 4$ and $n \neq 3$, one has $|3| = |5| = |6| = 1$. This statement should be restricted to $n = 4$. Indeed, since C is Class-I with $|1| = 1$, we have $|3| + |5| + |6| = n - 1$, and $|3|, |5|, |6|$ have the same parity. For $n = 2$, their sum would be 1, which is impossible under the common-parity condition; hence no such Class-I code exists. For $n = 4$, their sum is 3, so the common-parity condition implies $|3| = |5| = |6| = 1$. Thus the $n = 2$ case is vacuous, and no separate $\mathcal{Y}_3^A$ argument is needed. Accordingly, the previous discussion involving $|3| = 0$, $\{|5|, |6|\} = \{0, 1\}$ and `classI_count1_not_optimal` should be removed. The remark after Theorem 16 that code C' is linear is likewise not part of `class1_one`, which states only the strict or equal λ comparison.

The paper proves Theorem 16 when $|3| = \min\{|3|, |5|, |6|\}$ and, for the remaining argmin types 5 and 6, asserts that the code can be transformed to an equivalent one with $|3| = \min$, without giving the transformation. Lean supplies these reductions in `ClassI.lean`. When $|5| = \min$, the row swap $c_2 \leftrightarrow c_3$ (`swap12Code`) is a code equivalence that exchanges types 3 and 5 while fixing types 1 and 6, reducing to the $|3| = \min$ case (`class1_one_col5_strict`). When $|6| = \min$, the same reduction is also realizable by equivalence—flip every type-3/6 column and swap rows c_1 and c_3 (as in `swapRows02Code` in `Reduction.lean`).

The $|3| = \min$ branch of Theorem 16 uses four lemmas in `ClassI.lean` that characterize $\mathcal{Y}_3$ and $\mathcal{Y}_5$ and give closed forms for $\alpha_C^3(d)$ and $\alpha_C^5(d)$. In the paper these statements divide by 2, and the quotients are integral only under the Class-I parity condition $|3| \equiv |5| \equiv |6| \pmod{2}$. Lean restates them without divisions:

- `Y5_iff_weights_count1` (ll. 455–501): paper eq. (274), with the $|1| = 1$ simplifications (292)–(304). States $\mathcal{Y}_5$ membership by cleared weight inequalities only.
- `Y3_iff_weights_count1` (ll. 521–599): paper eq. (271), branches (281)–(287). Same $\mathcal{Y}_3$ split as the paper, with divisions removed and the boundary word assigned to the $\mathcal{Y}_3^A$

branch.

- `alpha5_closed_count1` (ll. 619–729): paper eq. (296). One triple sum over (w_3, w_5, w_6) with indicator `Y5WeightCond` and `dRow3WeightEq`.
- `alpha3_closed_count1` (ll. 751–867): paper eqs. (283) and (288). Merges the $\mathcal{Y}_3^A$ and $\mathcal{Y}_3^B$ sums into one triple sum with `Y3WeightCondA` $\vee$ `Y3WeightCondB` and `dRow2WeightEq`.

5 Final Remarks

We present a complete, kernel-checked Lean 4 formalization of the proofs from a research paper in coding theory. This note highlights the audit process, which is essential for ensuring the correctness of the formalization. Overall, with AI assistance, the formalization process can be completed without requiring significant human effort or expertise in formal mathematics.

Why formalization? Three lessons stand out during our experience. First, formalization gives a reproducible correctness check for a long, complex mathematical argument. Second, working at proof-script precision exposed imprecise hypotheses and gaps in the human-written proofs, leading to strengthened statements and proofs. Third, audited formalization scripts can be used to guide the AI to generate more accurate and efficient proofs, and even new results.

One area that has not been explored in this note, but is interesting, is the use of AI to assist in extending and generalizing the current results on finite-length codes. This could be a promising direction for future research.

References

- [1] T. Hales et al. “A Formal Proof of the Kepler Conjecture”. In: *Forum of Mathematics, Pi* 5 (2017). DOI: [10.1017/fmp.2017.1](https://doi.org/10.1017/fmp.2017.1).
- [2] G. Gonthier et al. “A Machine-Checked Proof of the Odd Order Theorem”. In: *Interactive Theorem Proving*. Vol. 7998. LNCS. Springer, 2013, pp. 163–179. DOI: [10.1007/978-3-642-39634-2_14](https://doi.org/10.1007/978-3-642-39634-2_14).
- [3] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik, and C. Szegedy. “Autoformalization with Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022. URL: <https://arxiv.org/abs/2205.12615>.
- [4] A. Agrawal, S. Gadgil, N. Goyal, A. Narayanan, and A. R. Tadipatri. *Towards a Mathematics Formalisation Assistant using Large Language Models*. 2022. arXiv: [2211.07524](https://arxiv.org/abs/2211.07524) [cs.CL]. URL: <https://arxiv.org/abs/2211.07524>.
- [5] R. Affeldt, C. Gagné, and D. Chafai. “Formalization of Shannon’s Theorems”. In: *Journal of Automated Reasoning* 53.4 (2014), pp. 337–364. DOI: [10.1007/s10817-013-9298-1](https://doi.org/10.1007/s10817-013-9298-1).

- [6] The mathlib Community. “The Lean Mathematical Library”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 2020, pp. 367–381. DOI: [10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824).
- [7] A. Florath. *Formal Foundations and Proof-Carrying Certificates for q-ary Covering Codes in Lean 4*. 2026. arXiv: [2606.09600](https://arxiv.org/abs/2606.09600) [cs.IT]. URL: <https://arxiv.org/abs/2606.09600>.
- [8] J.-H. Baek and J.-L. Kim. *Formalizing Building-Up Constructions of Self-Dual Codes through Isotropic Lines in Lean*. 2026. arXiv: [2604.08485](https://arxiv.org/abs/2604.08485) [cs.IT]. URL: <https://arxiv.org/abs/2604.08485>.
- [9] A. Evans and G. Angeris. *Succinct Proofs and Linear Algebra*. Cryptology ePrint Archive, Report 2023/1478. 2023. URL: <https://eprint.iacr.org/2023/1478>.
- [10] B. Kase. *SPLAT: Succinct Proofs and Linear Algebra (Lean 4 Formalization)*. 2024. URL: <https://github.com/bkase/splat>.
- [11] Math Inc. *ZkLinalg: Lean 4 Formalization of the FRI Protocol*. 2025. URL: <https://github.com/math-inc/ZkLinalg>.
- [12] Z-Tech. *sumcheck-lean4: Linear Codes and Mutual-Correlated Agreement*. 2025. URL: <https://github.com/z-tech/sumcheck-lean4>.
- [13] C. Zhu, Z. Tang, G. Zhen, Y. Cao, Y. Zhao, R. Chen, X. Zhao, L. Zhang, and X. Wang. *Lean-QIT: Towards a Formal Infrastructure for Quantum Information Theory*. 2026. arXiv: [2607.09632](https://arxiv.org/abs/2607.09632) [quant-ph]. URL: <https://arxiv.org/abs/2607.09632>.
- [14] L. de Moura and S. Ullrich. “The Lean 4 Theorem Prover and Programming Language”. In: *Automated Deduction—CADE 28*. Vol. 12699. LNCS. Springer, 2021, pp. 625–635. DOI: [10.1007/978-3-030-79876-3_37](https://doi.org/10.1007/978-3-030-79876-3_37).
- [15] Y. Dong and S. Yang. “On Optimal Finite-Length Block Codes of Size Four for Binary Symmetric Channels”. In: *IEEE Transactions on Information Theory* 71.1 (Jan. 2025), pp. 138–166. DOI: [10.1109/TIT.2024.3504823](https://doi.org/10.1109/TIT.2024.3504823).